



Hoja de Trabajo - Laboratorio No. 5

Nombre: Michael Darío Gutiérrez Quel **Carné:** 1199723 **Objetivo** Practica sobre gestión de archivos, jerarquía y formatos de almacenamiento, operaciones básicas de manejo de datos y el uso práctico de Git/GitHub/GitLab para control de versiones y trabajo colaborativo.

Serie I Responda cada enunciado de manera clara y breve.

1. Explicar la diferencia entre archivos físicos y archivos lógicos, y cómo se relacionan con la gestión de información.

Un archivo físico es el que está almacenado en un dispositivo, mientras que un archivo lógico es la forma en que un programa accede y maneja esa información.

2. Describir la jerarquía de memoria y mencionar al menos tres tipos de dispositivos de almacenamiento.

Va desde lo más rápido y caro hasta lo más lento y barato: registros del CPU → caché → RAM → almacenamiento secundario. Ejemplos de dispositivos de almacenamiento: disco duro (HDD), unidad de estado sólido (SSD) y memoria USB.

3. ¿Qué ventajas ofrece Git como sistema de control de versiones en proyectos de programación?

Permite llevar un historial completo de cambios, trabajar en paralelo con ramas sin confundir el trabajo, revertir errores fácilmente, y colaborar en equipo de forma ordenada

4. Listar y explicar dos diferencias entre archivos CSV y JSON, con ejemplos de uso.

CSV es texto plano separado por comas, ideal para tablas simples (ej. exportar datos de Excel). JSON tiene estructura jerárquica con llaves y valores, ideal para datos anidados (ej. una respuesta de una API). CSV no soporta bien datos anidados; JSON sí.

5. Explicar qué significa “abrir” y “cerrar” un archivo en programación, y dar un ejemplo en Python o Java.

Abrir es decirle al sistema operativo que vas a usar ese archivo (te da un "canal" de acceso). Cerrar libera ese recurso y guarda los cambios pendientes.

```
StreamReader sr = new StreamReader("archivo.txt"); string contenido = sr.ReadToEnd(); sr.Close();
```

6. ¿Qué es un repositorio remoto en GitHub y para qué sirve en el trabajo colaborativo?

Es una copia del proyecto guardada en la nube, que permite que varias personas trabajen sobre el mismo código y sincronicen sus cambios.

7. ¿Qué hace cada comando y qué diferencia existe entre `git fetch` y `git pull`?

`git fetch` trae los cambios del remoto, pero no los mezcla con tu código local (solo actualiza la información). `git pull` hace `fetch` + `merge` automáticamente, o sea, sí actualiza tu rama local.



8. Elaborar un cuadro comparativo entre almacenamiento estructurado y no estructurado, con ejemplos prácticos.

	Estructurado	No estructurado
Definición	Datos con formato fijo (tablas, columnas)	Sin formato definido
Ejemplo	Base de datos SQL, hoja de Excel	Videos, imágenes, correos, PDFs
Búsqueda	Fácil con consultas (SQL)	Más difícil, requiere procesamiento extra

9. Explicar qué hace el comando `git merge` y cómo se diferencia de `git rebase`.

`git merge` une dos ramas creando un commit nuevo que junta el historial de ambas (se ve como una "Y"). `git rebase` mueve los commits de una rama para que queden como si hubieran salido desde el punto actual de la otra rama (historial más limpio, en línea recta).

10. ¿Qué es un *fork* en GitHub y para qué sirve en el trabajo colaborativo?

Es una copia completa de un repositorio ajeno hacia tu propia cuenta. Sirve para proponer cambios a un proyecto sin tener permisos directos sobre el original (después mandas un Pull Request).

11. Listar y explicar dos diferencias entre archivos XML y JSON, con ejemplos de uso.

XML usa etiquetas de apertura y cierre (`<nombre>Luis</nombre>`), es más verboso. JSON usa llaves y comillas (`{"nombre": "Luis"}`), más compacto y liviano.

XML soporta atributos y validación con esquemas (XSD); JSON es más simple pero no tiene esa validación nativa.

12. Explicar la diferencia entre archivos secuenciales y archivos aleatorios, y cómo se utilizan en programación.

Los archivos secuenciales se leen de principio a fin, en orden (ej. leer un `.txt` línea por línea). Los archivos aleatorios permiten saltar directo a una posición específica sin leer todo lo anterior (ej. acceder al registro 500 de un archivo binario).



Serie II

Requerimiento: Realizar el algoritmo en pseudocódigo y diagrama de flujo para el siguiente requerimiento:

- Solicitar al usuario un archivo de texto.
- Leer su contenido y contar el número de líneas, palabras y caracteres.
- Mostrar los resultados en pantalla.

Consideraciones:

- El algoritmo debe incluir entradas, procesos y salidas claramente definidos.
- El diagrama de flujo debe representar cada paso de manera ordenada.

Serie II

Pseudocódigo:

INICIO

// ENTRADA

Escribir "Ingrese la ruta del archivo de texto:"

Leer ruta

Abrir archivo en modo lectura

lineas = 0

palabras = 0

caracteres = 0

// PROCESO

Mientras no sea fin de archivo:

 Leer línea

 lineas = lineas + 1

 caracteres = caracteres + longitud(línea)

 palabras = palabras + contar_palabras(línea) // separar por espacios

Fin mientras

Cerrar archivo

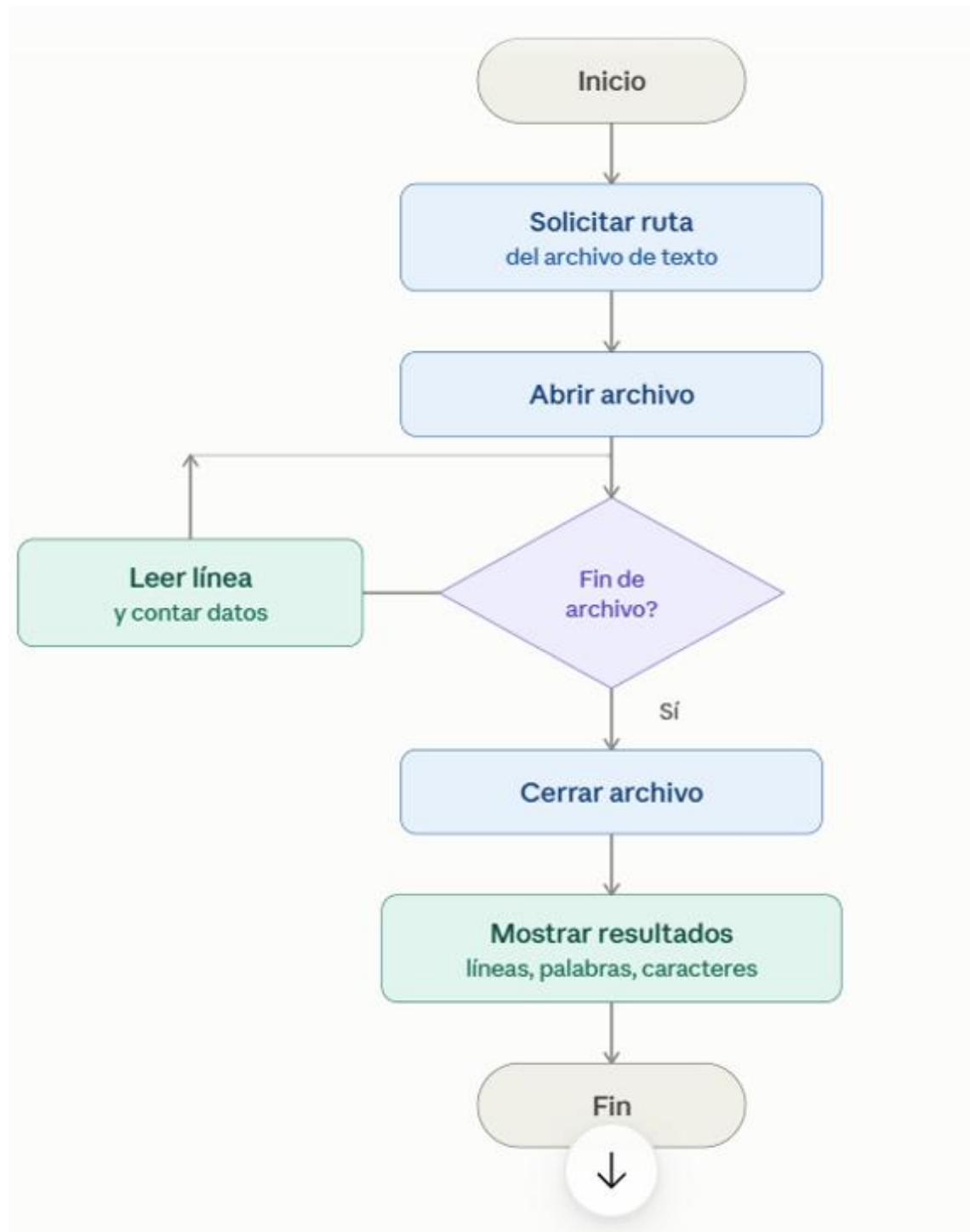
// SALIDA

Escribir "Líneas: ", lineas

Escribir "Palabras: ", palabras

Escribir "Caracteres: ", caracteres

FIN





Serie III

Requerimiento: Desarrollar un programa en lenguaje de programación **C# tipo consola** en el IDE de su preferencia. El programa debe:

- Solicitar al usuario su **nombre completo** (cada estudiante debe ingresar el suyo; en el ejemplo se usa *Luis Aguilar*).
- Solicitar la ruta de un archivo de texto, ubicado en: C:\MIA_Lab_5*<Nombre_Apellido>.txt*
- Leer el archivo y mostrar en pantalla: número de líneas, número de palabras y número de caracteres.
- Guardar los resultados en un archivo CSV dentro de la misma carpeta (C:\MIA_Lab_5) con el formato: *<Nombre_Apellido>,Lineas,Palabras,Caracteres*
- El archivo debe llamarse: *resultados_<Nombre_Apellido>.csv*

Ejemplo de salida:

Usuario: Luis Aguilar

Archivo: C:\MIA_Lab_5\Luis_Aguilar.txt

El archivo contiene: 120 líneas, 560 palabras, 3200 caracteres.

Resultados guardados en C:\MIA_Lab_5\resultados_Luis_Aguilar.csv

Consideraciones:

- Cada estudiante debe **renombrar el archivo de entrada** con su propio nombre.
- La codificación debe ser realizada en su totalidad por el alumno.
- No se permite utilizar funciones externas que automaticen el conteo.
- El estudiante debe demostrar dominio de lectura/escritura de archivos y uso de estructuras básicas.
- En comentarios dentro del código, denotar claramente:
 - **Entradas** (nombre del usuario, archivo de texto en C:\MIA_Lab_5).
 - **Proceso** (lectura, conteo de líneas, palabras y caracteres).
 - **Salidas** (impresión en consola y archivo CSV).

<https://github.com/Michael11997/url-mia-2026>

<https://github.com/Michael11997/url-mia-2026/pull/1>



Serie IV

Requerimiento: Simular un flujo de trabajo colaborativo

Pasos a seguir:

1. Clonar su repositorio MIA de GitHub.
2. Crear una rama con el nombre: `feature/[nombre_apellido]_Lab_5`
Ejemplo: `feature/Luis_Aguilar_Lab_5`
3. Dentro del repositorio, crear la carpeta: `MIA_LAB_5`
4. Subir dentro de esa carpeta:
 - **PDF:** Serie II: Algoritmo en pseudocódigo y diagrama de flujo,
 - **Archivo .cs:** subir únicamente el archivo **Program.cs** (la clase principal con el Main)
 - **Archivos de prueba:** Archivos: `.txt` y `.csv` de serie III
5. Realizar un commit siguiendo la convención: “feat: agrega soluciones del Lab_5_ [nombre apellido]”
6. Publicar la rama en el repositorio remoto.
7. Crear un Pull Request desde la rama `feature/[nombre_apellido]_Lab_5` hacia la rama `dev`.
 - Título: feat: agrega soluciones del Lab_5 1
 - Descripción: explicar qué contiene la carpeta `MIA_LAB_5` (Serie I en PDF, Serie II y Serie III).

Entrega: Link del Pull Request creado en su repositorio remoto github